

Amrita School of Computing

23CSE211

**Design and Analysis
of Algorithms**

WEEK – 4

NAME : SRINIVAS R

ROLL : CH.SC.U4CSE24146

1. AVL TREE

Balance Factor = Height(left subtree) – Height(right subtree).

The elements

157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133

Code :

```
//ch.sc.u4cse24146
```

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct AvlNode{  
    int data;  
    struct AvlNode* left;  
    struct AvlNode* right;  
    int height;  
};
```

```
int height(struct AvlNode* root){  
    if(root==NULL){  
        return 0;  
    }  
    else{  
        return root->height;  
    }  
}
```

```
int max(int a, int b){  
    return (a>b)?a:b;  
}
```

```
struct AvlNode* leftRotate(struct AvlNode* y){  
    struct AvlNode* x = y->right;
```

```

    struct AvlNode* T2 = x->left;
    x->left = y;
    y->right = T2;
    y->height = max(height(y->left),height(y->right))+1;
    x->height = max(height(x->left),height(x->right))+1;
    return x;
}

```

```

struct AvlNode* rightRotate(struct AvlNode* y){
    struct AvlNode* x = y->left;
    struct AvlNode* T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = max(height(y->left),height(y->right))+1;
    x->height = max(height(x->left),height(x->right))+1;
    return x;
}

```

```

int balancing(struct AvlNode* root){
    if(root==NULL){
        return 0;
    }
    else{
        return (height(root->left)-height(root->right));
    }
}

```

```

struct AvlNode* insert(struct AvlNode* root,int data){
    if(root==NULL){
        struct AvlNode* root = (struct AvlNode*)malloc(sizeof(struct
AvlNode));
        root->left = NULL;
        root->right = NULL;
        root->data = data;
        root->height = 1;
    }
}

```

```

        return root;
    }
    if(data<root->data){
        root->left = insert(root->left,data);
    }
    else if(data>root->data){
        root->right = insert(root->right,data);
    }
    root->height = max(height(root->left),height(root->right))+1;

    int bf = balancing(root);
    if(bf>1 && data < root->left->data){
        return rightRotate(root);
    }
    if(bf<-1 && data>root->right->data){
        return leftRotate(root);
    }
    if(bf>1 && data > root->left->data){
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
    if(bf<-1 && data < root->right->data){
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }

    return root;
}

struct AvlNode* maxchild(struct AvlNode* root){
    if(root == NULL || root->right ==NULL){
        return root;
    }
    else{
        return (maxchild(root->right));
    }
}

```

```
}  
}
```

```
struct AvlNode* deleteA(struct AvlNode* root,int data){  
    if(root==NULL){  
        return NULL;  
    }  
    struct AvlNode* temp;  
    if(data < root->data){  
        root->left = deleteA(root->left,data);  
    }  
    else if(data > root->data){  
        root->right = deleteA(root->right,data);  
    }  
    else{  
        // Element found  
        if(root->right && root->left){  
            temp = maxchild(root->left);  
            root->data = temp->data;  
            root->left = deleteA(root->left,temp->data);  
        }  
        else {  
            // Node with zero or one child  
            temp = root;  
            if (root->left == NULL) {  
                root = root->right;  
            }  
            else if (root->right == NULL) {  
                root = root->left;  
            }  
            free(temp);  
        }  
    }  
    if (root == NULL)
```

```

        return NULL;

    root->height = max(height(root->left),height(root->right))+1;

    int bf = balancing(root);

    if(bf>1 && balancing(root->left)>=0){//no right child (heavy left)
        return rightRotate(root);
    }
    if(bf<-1 && balancing(root->right)<0){//only right (heavy right)
        return leftRotate(root);
    }
    if(bf>1 && balancing(root->left)<0){
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
    if(bf<-1 && balancing(root->right)>=0){
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }
    return root;
}

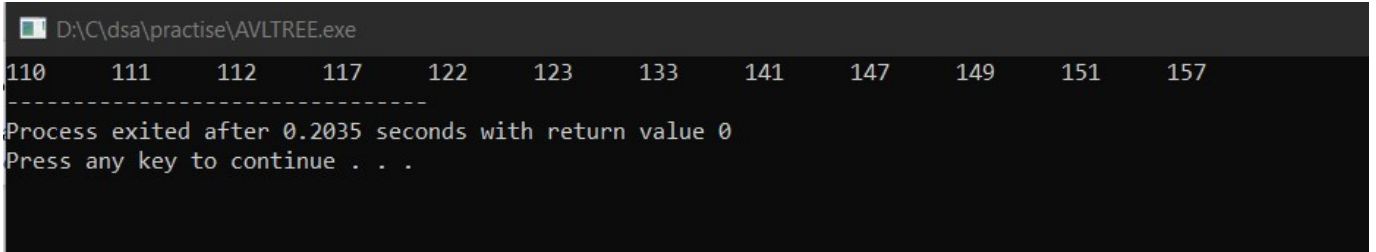
void display(struct AvlNode* root){
    if(root==NULL){
        return;
    }
    else{
        display(root->left);
        printf("%d\t",root->data);
        display(root->right);
    }
}

int main(){

```

```
struct AvlNode* root = NULL;
root=insert(root,157);
root=insert(root,110);
root=insert(root,147);
root=insert(root,122);
root=insert(root,111);
root=insert(root,149);
root=insert(root,151);
root=insert(root,141);
root=insert(root,123);
root=insert(root,112);
root=insert(root,117);
root=insert(root,133);
display(root);
}
```

Output :



```
D:\C\dsa\practise\AVLTREE.exe
110    111    112    117    122    123    133    141    147    149    151    157
-----
Process exited after 0.2035 seconds with return value 0
Press any key to continue . . .
```

2. RED – BLACK TREE

Code :

```
#include <stdio.h>
#include <stdlib.h>

#define RED 1
#define BLACK 0

struct Node {
    int data;
    int color;
    struct Node *left, *right, *parent;
};

struct Node* root = NULL;

struct Node* newNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->color = RED;
    node->left = node->right = node->parent = NULL;
    return node;
}

void leftRotate(struct Node* x) {
    struct Node* y = x->right;
    x->right = y->left;

    if (y->left)
        y->left->parent = x;

    y->parent = x->parent;

    if (!x->parent)
        root = y;
    else if (x == x->parent->left)
        x->parent->left = y;
    else
        x->parent->right = y;

    y->left = x;
    x->parent = y;
}

void rightRotate(struct Node* y) {
    struct Node* x = y->left;
    y->left = x->right;

    if (x->right)
```



```

    x->right->parent = y;

x->parent = y->parent;

if (!y->parent)
    root = x;
else if (y == y->parent->left)
    y->parent->left = x;
else
    y->parent->right = x;

x->right = y;
y->parent = x;
}

void fixInsert(struct Node* z) {
    while (z->parent && z->parent->color == RED) {

        if (z->parent == z->parent->parent->left) {
            struct Node* y = z->parent->parent->right;

            if (y && y->color == RED) {
                z->parent->color = BLACK;
                y->color = BLACK;
                z->parent->parent->color = RED;
                z = z->parent->parent;
            } else {
                if (z == z->parent->right) {
                    z = z->parent;
                    leftRotate(z);
                }
                z->parent->color = BLACK;
                z->parent->parent->color = RED;
                rightRotate(z->parent->parent);
            }
        } else {
            struct Node* y = z->parent->parent->left;

            if (y && y->color == RED) {
                z->parent->color = BLACK;
                y->color = BLACK;
                z->parent->parent->color = RED;
                z = z->parent->parent;
            } else {
                if (z == z->parent->left) {
                    z = z->parent;
                    rightRotate(z);
                }
                z->parent->color = BLACK;
                z->parent->parent->color = RED;
                leftRotate(z->parent->parent);
            }
        }
    }
}

```

```

    }
}
}
root->color = BLACK;
}

void insert(int data) {
    struct Node* z = newNode(data);
    struct Node* y = NULL;
    struct Node* x = root;

    while (x) {
        y = x;
        if (z->data < x->data)
            x = x->left;
        else
            x = x->right;
    }

    z->parent = y;

    if (!y)
        root = z;
    else if (z->data < y->data)
        y->left = z;
    else
        y->right = z;

    fixInsert(z);
}

void inorder(struct Node* node) {
    if (node) {
        inorder(node->left);
        printf("%d ", node->data);
        inorder(node->right);
    }
}

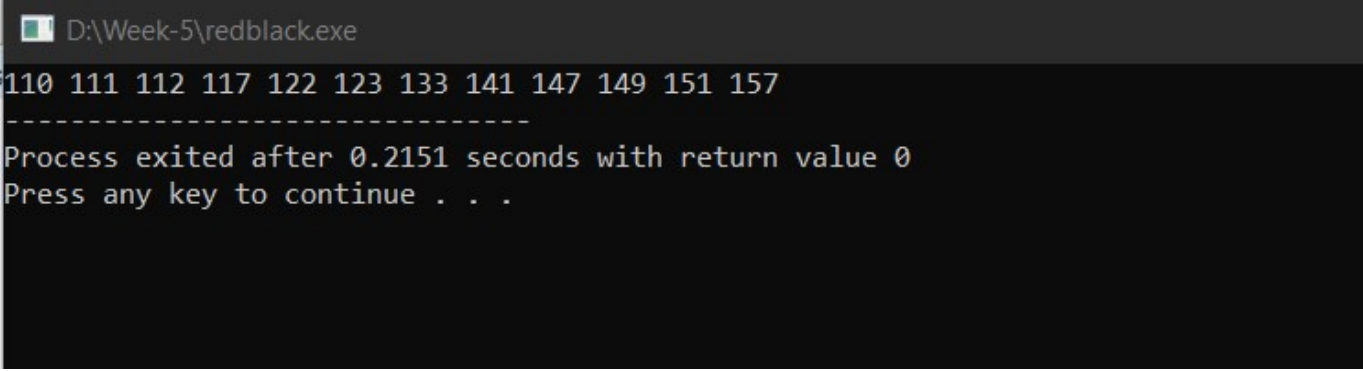
int main() {
    int arr[] = {157,110,147,122,111,149,151,141,123,112,117,133};
    int n = 12;

    for (int i = 0; i < n; i++)
        insert(arr[i]);

    inorder(root);
    return 0;
}

```

Output :



A screenshot of a Windows command prompt window. The title bar at the top reads "D:\Week-5\redblack.exe". The command prompt shows a list of prime numbers: 110 111 112 117 122 123 133 141 147 149 151 157. Below this list is a dashed line. The next line of output states: "Process exited after 0.2151 seconds with return value 0". The final line of output is: "Press any key to continue . . .".

```
D:\Week-5\redblack.exe
110 111 112 117 122 123 133 141 147 149 151 157
-----
Process exited after 0.2151 seconds with return value 0
Press any key to continue . . .
```